DataSynth: A Comprehensive Framework for Generating Realistic Synthetic Enterprise Accounting Data with Provable Statistical Properties

Michael Ivertowski
Assurance DID Research
Ernst & Young Ltd. Switzerland
michael.ivertowski@ch.ey.com
https://orcid.org/0009-0008-7829-2249

March 2, 2026

Abstract

Synthetic data generation for enterprise accounting and financial auditing remains a critical bottleneck for machine-learning research, fraud-detection system development, and regulatory compliance testing. Existing approaches either produce statistically naïve tabular data that fails basic accounting invariants or require access to sensitive production datasets that cannot be shared. We present DataSynth, an open-source framework that generates *end-to-end coherent* synthetic enterprise data spanning journal entries, document flows, subledgers, financial statements, intercompany eliminations, and process-mining event logs—all from a single declarative YAML configuration.

DataSynth makes five main contributions. **(i)** A *statistically grounded generation engine* built on log-normal mixture models, five copula families (Gaussian, Clayton, Gumbel, Frank, Student-t), and Benford's-law–compliant amount sampling that preserves cross-field correlations and marginal properties simultaneously. **(ii)** A *multi-stage anomaly injection framework* with 33 anomaly types organized across five categories (fraud, error, process, statistical, relational), including progressive fraud schemes with state-machine advancement, configurable difficulty, severity, and confidence scoring. **(iii)** A *privacy-preserving fingerprint module* that extracts distributional summaries from real datasets under differential privacy (ε-DP) and k-anonymity, enabling synthetic data that mirrors production characteristics without exposing individual records. **(iv)** A *comprehensive evaluation suite* comprising 15 domain-specific coherence validators, Benford and distribution-fit tests, temporal-pattern analysis, drift detection, and ML-readiness metrics including graph-neural-network feature assessment. **(v)** A *counterfactual simulation engine* with causal DAG propagation for what-if analysis, enabling controlled interventions on financial process dependencies and assessment of downstream impacts.

Implemented in Rust across 15 crates totaling over 100 k lines of code, DataSynth achieves throughput exceeding 2×10^5 journal entries per second on a single core with deterministic reproducibility via ChaCha8-seeded RNG. The framework supports five accounting frameworks (US GAAP, IFRS, French GAAP, German GAAP, and dual-reporting) and multi-period stateful generation with checkpoint support for long-horizon simulations.

We demonstrate that synthetic datasets generated by DataSynth satisfy the accounting equation, preserve Benford's-law compliance (MAD < 0.006), maintain document-chain referential integrity, and support downstream fraud-detection models that achieve within 3% of the F1 scores obtained on real data.

Keywords: synthetic data generation, enterprise accounting, fraud detection, Benford's law, copula models, differential privacy, process mining, graph neural networks, audit analytics, counterfactual analysis, causal inference

1 Introduction

The digitization of enterprise resource planning (ERP) systems has produced vast repositories of financial transaction data that underpin modern accounting, auditing, and regulatory compliance. Machine-learning approaches to fraud detection [12, 33], continuous auditing [32], and anomaly analytics [27] have shown considerable promise, yet they face a persistent data-access challenge: real accounting data is proprietary, regulated under frameworks such as SOX [31] and GDPR [8], and rarely available for open research.

Synthetic data generation offers a path forward, but existing approaches fall short in several dimensions:

1. **Statistical fidelity.** General-purpose tabular generators (GANs, VAEs, CTGAN [35]) do not enforce domain-specific invariants such as balanced double-entry bookkeeping, Benford's-law compliance [2, 24], or accounting equation coherence ($A = L + E$).

2. **Relational completeness.** Enterprise data is inherently multi-table and multi-process. A purchase order triggers goods receipts, vendor invoices, and payments; these in turn affect subledgers, trial balances, and financial statements. Generating tables in isolation destroys these referential chains.

3. **Anomaly realism.** Injecting fraud and error patterns post hoc often produces anomalies that are trivially detectable or statistically implausible, limiting the usefulness of generated data for training robust detection models.

4. **Privacy guarantees.** When real data characteristics must be preserved, naïve summary statistics may leak sensitive information. Formal privacy mechanisms are needed to bridge the gap between real-data fidelity and confidentiality.

Contributions. We present DataSynth, a comprehensive open-source framework that addresses all four challenges simultaneously. Our contributions are:

- A **layered generation architecture** (Section 3) comprising 15 Rust crates that separate domain models, statistical distributions, business-process generators, output sinks, and evaluation into composable modules.

- A **statistically grounded sampling engine** (Section 4) featuring log-normal mixture models, five copula families for dependency modeling, and first- and second-digit Benford compliance with provable mean-absolute-deviation bounds.

- A **multi-process generation pipeline** (Section 5) that produces coherent data spanning Procure-to-Pay (P2P), Order-to-Cash (O2C), Record-to-Report (R2R), Source-to-Contract (S2C), hire-to-retire (HR), and manufacturing processes, with three-way matching, inter-company elimination, and period-close procedures.
- A **multi-stage anomaly injection framework** (Section 6) with 33 anomaly types, progressive fraud schemes, temporal clustering, cascade propagation, difficulty scoring, and ground-truth labels suitable for supervised and semi-supervised learning.
- A **privacy-preserving fingerprint module** (Section 9) that extracts distributional profiles under ε-differential privacy and k-anonymity, enabling generation of synthetic data that mirrors real-world characteristics without exposing individual records.
- A **comprehensive evaluation suite** (Section 8) with 15 coherence validators, statistical goodness-of-fit tests, ML-readiness metrics, and an auto-tuning engine that generates configuration patches from evaluation gaps.
- A **counterfactual simulation engine** (Section 7) built on a causal directed acyclic graph (DAG) with 8 transfer function types, enabling what-if analysis of financial process dependencies under controlled interventions.
- **Multi-GAAP support** for five accounting frameworks (US GAAP, IFRS, French GAAP, German GAAP, and dual-reporting) with 7 country packs providing jurisdiction-specific tax, calendar, and banking configurations.

Empirical grounding. Our design is informed by a large-scale empirical study of 155 real-world ISO 21378:2019–compliant general ledger datasets comprising 364 million journal entries with 2.4 billion line items across all industry sectors [17]. This analysis reveals the distributional characteristics of production accounting data—e.g., that 92.9% of entries have 2–9 line items with extreme concentration at 2-line entries (60.7%), and that chart-of-accounts complexity spans 76 to 2,536 accounts—which directly calibrate DataSynth's generation models (Section 10.2).

Scope and audience. DataSynth targets three communities: (a) ML researchers needing realistic labeled accounting datasets for fraud detection and anomaly analytics, (b) auditors and regulators seeking synthetic test environments for continuous auditing tools, and (c) ERP vendors requiring high-volume test data for system integration and stress testing.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 presents the system architecture. Section 4 formalizes the statistical foundations. Section 5 details the generation pipeline. Section 6 describes the anomaly injection framework. Section 8 covers the evaluation suite. Section 9 discusses the privacy-preserving fingerprint module. Section 7 introduces the counterfactual simulation engine. Section 10 reports experimental results. Section 11 concludes.

2 Related Work

We position DataSynth at the intersection of four research streams: synthetic tabular data generation, accounting-domain data synthesis, anomaly injection for fraud detection, and privacy-preserving data sharing.

2.1 General-Purpose Tabular Data Generation

Deep generative models have become the dominant paradigm for synthetic tabular data. CTGAN [35] introduced mode-specific normalization and conditional training for mixed-type columns. TVAE [35] applies variational autoencoders with the same preprocessing. Subsequent work has refined these approaches: CopulaGAN [25] combines copula transformations with GAN training, REaLTabFormer [30] leverages autoregressive transformers for relational tables, and TabDDPM [20] adapts diffusion models to tabular settings.

While these methods excel at preserving marginal distributions and pairwise correlations, they treat tables as independent collections of rows and do not enforce cross-table referential integrity, temporal ordering constraints, or domain-specific algebraic invariants (e.g., balanced entries). DataSynth addresses this gap by embedding accounting axioms directly into the generation process.

2.2 Accounting-Domain Data Synthesis

Domain-specific synthetic data for accounting has received comparatively little attention. The GL.ai dataset [28] provides federated synthetic journal entries but lacks document-flow linkage and subledger coherence. The SAFD (Simulated Accounting Fraud Dataset) [29] generates entries with injected anomalies, yet uses uniform random amounts that violate Benford's law. BankSim [22] focuses narrowly on payment transactions for fraud research.

In contrast, DataSynth generates a full enterprise data stack: chart of accounts, master data, multi-process document flows with three-way matching, subledgers, trial balances, financial statements, intercompany eliminations, period-close entries, and OCEL 2.0 event logs—all mutually consistent.

2.3 Anomaly Injection and Fraud Simulation

Fraud detection benchmarks typically rely on post-hoc label injection or manual scenario creation. The IEEE-CIS fraud dataset [13] provides real transaction labels but lacks accounting context. Pourhabibi et al. [26] survey fraud detection methods and note the scarcity of labeled multi-dimensional accounting fraud data. Dbouk and Jamali [6] inject statistical anomalies into time series but do not model multi-stage fraud schemes that evolve over time.

DataSynth introduces a *scheme-based* injection model where fraud evolves through state-machine stages (e.g., vendor kickback schemes that escalate over months), producing temporally correlated, multi-transaction anomaly patterns with ground-truth difficulty and severity labels.

2.4 Privacy-Preserving Data Sharing

Differential privacy [7] provides formal guarantees for statistical queries. DPGAN [34] and PATE-GAN [18] train generative models under DP constraints but are computationally expensive and may degrade statistical fidelity. The synthetic data vault (SDV) [25] framework supports metadata-driven generation but does not provide formal privacy guarantees on the metadata itself.

DataSynth's fingerprint module takes a different approach: it extracts distributional summaries (not raw data) under ε-DP with explicit ε-budget tracking and k-anonymity suppression, producing a compact `.dsf` archive from which a fully configured generator can be instantiated. This decouples the privacy boundary (extraction) from the generation process (synthesis), enabling auditability and composability.

2.5 Process Mining and Event Logs

Object-centric event logs (OCEL 2.0) [10] generalize traditional single-case-ID event logs by associating events with multiple typed objects. While tools such as PM4Py [4] support OCEL 2.0 import/export, generating realistic synthetic OCEL 2.0 logs with consistent underlying business data has not been addressed. DataSynth produces OCEL 2.0 event logs that are traceable to the same journal entries, document flows, and master-data records that constitute the rest of the synthetic dataset.

2.6 Accounting Network Construction

The representation of double-entry bookkeeping as directed graphs dates back to Ijiri [14], who established that journal entries have a natural graph interpretation: accounts are nodes, and debit/credit flows form directed edges. Fellingham [9] provides a modern algebraic treatment of double-entry systems. Arya et al. [1] study the problem of inferring individual transactions from aggregated financial statements, highlighting the combinatorial complexity of reconstructing edge-level flows from node-level balances.

Recent work has revived interest in accounting graphs for audit and fraud detection. Guo et al. [11] demonstrate that accounting graph topology—node degree distributions, clustering coefficients, and community structure—can discriminate between normal and fraudulent bookkeeping patterns. Boersma [5] applies complex network analysis to audit data, modeling interfirm relationships and transaction patterns as multi-layer graphs. Liang et al. [21] develop pattern recognition and anomaly detection methods directly on bookkeeping graph representations.

Ivertowski et al. [17] present a hardware-accelerated approach for constructing accounting networks from general ledger data, analyzing 155 real-world ISO 21378:2019–compliant datasets comprising approximately 364 million journal entries with 2.4 billion line items. Their analysis reveals key distributional characteristics of production accounting data (see Section 10.2 for details), which directly inform DataSynth's generation model.

While these approaches focus on *analyzing* real accounting data as graphs, none address the complementary problem of *generating* realistic synthetic accounting data that exhibits the same graph-theoretic properties. DataSynth bridges this gap: the graph export module

Table 1: Feature comparison with related systems. Columns: **Bal** = balanced entries, **Ben** = Benford compliance, **Doc** = document-flow chains, **Sub** = subledger coherence, **FS** = financial statements, **Anom** = anomaly injection with labels, **Priv** = formal privacy guarantees, **OCEL** = process mining logs, **Graph** = graph export for GNNs.

System	Bal	Ben	Doc	Sub	FS	Anom	Priv	OCEL	Graph
CTGAN / TVAE	–	–	–	–	–	–	–	–	–
SDV / CopulaGAN	–	–	✓*	–	–	–	–	–	–
REaLTabFormer	–	–	✓	–	–	–	–	–	–
GL.ai	✓	–	–	–	–	✓	✓*	–	–
BankSim	–	–	–	–	–	✓	–	–	–
SAFD	✓	–	–	–	–	✓	–	–	–
ACA / Guo et al.	–	–	–	–	–	–	–	–	✓*
DataSynth	✓	✓	✓	✓	✓	✓	✓	✓	✓

*Partial support.

```
Layer 4 Analytics  Export datasynth-eval | datasynth-graph |
datasynth-fingerprint
Layer 3 Domain Generators datasynth-generators | datasynth-banking |
datasynth-ocpm datasynth-standards
Layer 2 Orchestration  Output datasynth-runtime | datasynth-output |
datasynth-config
Layer 1 Foundation datasynth-core (models, distributions, resource guards)
datasynth-test-utils
```

Figure 1: Layered crate architecture of DataSynth. Dependency arrows point downward; each layer depends only on layers below it.

(Section 5.6) produces transaction networks whose structural characteristics are calibrated against the real-world distributions reported in the literature.

2.7 Summary of Gaps

Table 1 summarizes feature coverage across related systems.

3 System Architecture

DataSynth is implemented as a Rust workspace comprising 15 crates organized into five architectural layers (Figure 1). The layered design enforces separation of concerns: domain models and statistical primitives live in the foundation layer, generators in the process layer, and evaluation, privacy, and export capabilities in the analytics layer.

3.1 Layered Crate Organization

Layer 1: Foundation. `datasynth-core` defines over 120 domain model types (journal entries, master data, document flows, subledgers, financial statements, controls, relationships), statistical distribution samplers (Benford, copula, mixture, Pareto, Weibull, Beta, zero-inflated), and resource-management primitives (memory guard, CPU monitor, disk guard, degradation

controller). All models use `rust_decimal::Decimal` for exact arithmetic, eliminating IEEE 754 rounding errors in financial calculations.

Layer 2: Orchestration. `datasynth-runtime` houses the `GenerationOrchestrator` that coordinates the full generation workflow: configuration loading, company initialization, generator sequencing, anomaly injection, evaluation, and output. `datasynth-config` provides schema validation, industry presets, and complexity-level mappings. `datasynth-output` implements streaming sinks for CSV, JSON, Parquet, and Apache Arrow formats.

Layer 3: Domain Generators. `datasynth-generators` contains 50+ specialized generators organized by business process: P2P document flows, O2C chains, S2C sourcing, HR/payroll, manufacturing, period-close, intercompany, and anomaly injection. `datasynth-banking` adds KYC/AML-specific generation with five money laundering typologies. `datasynth-ocpm` produces OCEL 2.0 compliant event logs. `datasynth-standards` implements five accounting frameworks (US GAAP, IFRS, French GAAP/PCG, German GAAP/HGB, and dual-reporting) together with ISA, SOX, and PCAOB audit standard models.

Layer 4: Analytics. `datasynth-eval` provides the evaluation framework (15 coherence validators, statistical tests, ML-readiness metrics, auto-tuner). `datasynth-graph` exports heterogeneous transaction graphs in PyTorch Geometric, Neo4j, and DGL formats. `datasynth-fingerprint` implements privacy-preserving extraction and synthesis.

Layer 5: Interfaces. A CLI binary (`datasynth-data`) supports `generate`, `validate`, `init`, `info`, and `fingerprint` subcommands. A REST/gRPC/WebSocket server enables streaming generation with API-key authentication and rate limiting. A Tauri + SvelteKit desktop UI provides visual configuration and monitoring.

3.2 Deterministic Reproducibility

All randomness flows from a single `u64` seed through the ChaCha8 cryptographic PRNG [3]. Each generator receives a derived seed computed as $\texttt{base_seed} + \texttt{offset}(g)$ where $\texttt{offset}(g)$ is a fixed constant per generator type g. This guarantees:

1. **Bitwise reproducibility:** identical seed $\Rightarrow$ identical output across runs and platforms.
2. **Generator independence:** modifying one generator's parameters does not perturb another's output stream.
3. **Parallel safety:** each thread maintains a local RNG instance; no global state is shared.

3.3 Collision-Free UUID Generation

Synthetic records require globally unique identifiers that are deterministic yet collision-free across generator types. DataSynth employs an FNV-1a hash-based UUID factory with a 16-byte structure:

$$\underbrace{\texttt{seed}[0{:}5]}_{6\text{ bytes}} \parallel \underbrace{\texttt{gen_type}}_{1\text{ byte}} \parallel \underbrace{\texttt{0x4}\|\texttt{sub}}_{1\text{ byte}} \parallel \underbrace{\texttt{counter}}_{8\text{ bytes}} \tag{1}$$

The `gen_type` discriminator byte (39 values, `0x01`–`0x3E`) partitions the UUID space by generator, ensuring that journal-entry IDs can never collide with vendor IDs or document-flow IDs regardless of counter values. The counter uses `AtomicU64` with relaxed ordering for thread-safe monotonic increment.

3.4 Resource Management and Graceful Degradation

Large-scale generation runs may stress system resources. DataSynth implements a unified resource guard that monitors memory (via `/proc/self/statm` on Linux), disk space (via `statvfs`), and CPU utilization with configurable thresholds. When thresholds are breached, the system transitions through four degradation levels:

Level	**Memory**	**Batch Size**	**Features Disabled**
Normal	$< 70\%$	100%	None
Reduced	$70{-}85\%$	50%	Data quality injection
Minimal	$85{-}95\%$	25%	Anomaly injection
Emergency	$> 95\%$	Flush	Optional fields

A 5% hysteresis band prevents oscillation between levels. The system logs all transitions and resumes full generation automatically when resources recover.

3.5 Configuration System

DataSynth uses a hierarchical YAML configuration with 30+ sections covering global settings, companies, chart of accounts, transactions, document flows, distributions, temporal patterns, anomaly injection, internal controls, intercompany, banking, standards, and more. Industry presets (manufacturing, retail, financial services, healthcare, technology) provide complete default configurations. Three complexity levels (small $\sim$100 accounts, medium $\sim$400, large $\sim$2500) control chart-of-accounts granularity and transaction volume.

Configuration validation enforces invariants such as ascending approval thresholds, distribution weights summing to 1.0 (± 0.01), period lengths in $[1, 120]$ months, and rates in $[0, 1]$.

Multi-GAAP Framework. DataSynth supports framework-aware generation across five accounting standards: US GAAP, IFRS, French GAAP (Plan Comptable Général), German GAAP (HGB), and dual-reporting mode. Each framework determines GL account numbering conventions (e.g., SKR04 for German, PCG classes for French), account classification rules, and depreciation methods. Seven country packs (FR, JP, CN, IN, IT, ES, CA) bundle currency, tax regime, fiscal-year convention, and banking configurations into reusable presets. German jurisdictions additionally support GoBD-compliant audit export, and French jurisdictions produce FEC (*Fichier des Écritures Comptables*) extracts, satisfying the structured data access requirements of their respective tax authorities.

3.6 Streaming Pipeline

DataSynth supports phase-aware streaming output via the `PhaseSink` trait, which defines a uniform interface for emitting generated records as they are produced rather than buffering entire datasets in memory. Three sink implementations are provided: a file sink that writes JSONL (newline-delimited JSON), an HTTP sink that pushes records to a configurable endpoint, and a no-op sink for benchmarking. Each sink tracks items emitted, bytes sent, and generation phases completed, enabling real-time consumption of synthetic data by downstream systems and progress monitoring by the orchestrator.

3.7 Multi-Period Session Management

Enterprise datasets often span multiple fiscal years, with each period building on the prior period's closing balances and entity state. DataSynth addresses this with `GenerationSession`, a stateful session manager that persists its state to `.dss` checkpoint files between runs. A `GenerationPeriod` splitter divides the configured total timespan into fiscal-year-aligned periods. `SessionState` tracks RNG seeds, account balance snapshots, document-ID counters, and entity counts across periods, enabling incremental generation via `--append --months N`. Deterministic seed advancement ($s_{t+1} = f(s_t, t)$) ensures that multi-period runs produce bitwise-identical output regardless of whether they execute in a single invocation or across multiple incremental sessions.

3.8 Performance

Versions 0.9–0.11 introduced several throughput optimizations. Line-item collections use `SmallVec<[T; 4]>` to avoid heap allocation for typical entry sizes. Temporal CDF tables are precomputed and cached, and company selection uses binary search over cumulative weights. Integer and floating-point formatting use the `itoa` and `ryu` crates, respectively, bypassing `std::fmt`. Output compression uses `zstd` at configurable levels (1–9).

For multi-core generation, the `ParallelGenerator` trait splits the seed space deterministically across threads, so that the union of per-thread outputs is identical to a single-threaded run with the same seed. Master data and journal-entry generation are fully parallelized under this scheme, achieving near-linear scaling up to 8 cores. Collectively these changes yield approximately 2× single-threaded throughput, exceeding 2×10^5 journal entries per second on a modern x86-64 core.

4 Statistical Foundations

This section formalizes the statistical machinery underlying DataSynth's data generation. We cover Benford-compliant amount sampling, copula-based dependency modeling, mixture distributions, and temporal dynamics.

4.1 Benford's Law Compliance

Benford's law [2, 24] states that the leading digit $d \in \{1, \ldots, 9\}$ of naturally occurring numerical data follows the distribution:

$$P(D_1 = d) = \log_{10}\left(1 + \frac{1}{d}\right). \tag{2}$$

The generalization to the first two digits $d_1 d_2 \in \{10, \ldots, 99\}$ is:

$$P(D_{12} = d_1 d_2) = \log_{10}\left(1 + \frac{1}{d_1 \cdot 10 + d_2}\right). \tag{3}$$

DataSynth implements three Benford-aware samplers:

BenfordSampler. Uses CDF-based inverse-transform sampling over the first-digit probabilities. Given a pre-computed CDF vector $F = [0.30103, 0.47712, 0.60206, \ldots, 1.0]$, a uniform draw $u \sim \mathcal{U}(0, 1)$ is mapped to digit d via binary search: $d = \min\{k : F[k] \geq u\}$. The magnitude (order of magnitude) is sampled adaptively from a configured range, and trailing digits are drawn uniformly.

EnhancedBenfordSampler. Extends first-digit sampling to the first-two-digit distribution (Equation (3)), providing finer-grained statistical fidelity. This is critical for passing second-digit tests commonly used in forensic accounting [24].

BenfordDeviationSampler. Generates amounts that *intentionally violate* Benford's law for anomaly injection. Five deviation modes are supported:

- **RoundNumberBias**: Over-represents amounts ending in multiples of powers of ten (e.g., \$1,000, \$5,000).
- **ThresholdClustering**: Concentrates amounts just below approval thresholds (e.g., \$4,999 for a \$5,000 limit).
- **UniformFirstDigit**: Replaces Benford with $\mathcal{U}\{1, 9\}$.
- **DigitBias**: Over-weights specific digits (anti-Benford probabilities emphasize 5, 7, 9).
- **TrailingZeros**: Forces amounts to have zero trailing digits.

Proposition 4.1 (MAD bound)**.** *Let $\hat{p}_d$ denote the empirical first-digit frequency from n samples of the BenfordSampler. The mean absolute deviation* $\text{MAD} = \frac{1}{9}\sum_{d=1}^{9} |\hat{p}_d - p_d|$ *satisfies* $\mathbb{E}[\text{MAD}] \leq \frac{1}{9}\sqrt{\frac{\pi}{2n}}$ *for large n, ensuring convergence to "close conformity" (MAD < 0.006) as classified by Nigrini [24] for $n \gtrsim 7{,}000$.*

4.2 Copula-Based Dependency Modeling

Real accounting data exhibits complex dependencies: transaction amounts correlate with line-item counts, approval levels depend on amounts, and payment timing relates to invoice size. DataSynth separates marginal distributions from dependency structure using copulas [23].

Table 2: Copula properties and recommended accounting applications.

Copula	Lower Tail	Upper Tail	Use Case
Gaussian	None	None	General correlation
Clayton	Yes	None	Payment defaults, distress
Gumbel	None	Yes	High-value co-occurrence
Frank	None	None	Symmetric, no extremes
Student-t	Yes	Yes	Joint tail events

Definition 4.2 (Copula)**.** *A d-dimensional copula $C : [0,1]^d \to [0,1]$ is a joint CDF with uniform marginals. By Sklar's theorem, any joint distribution H with marginals $F_1, \ldots, F_d$ can be written as $H(x_1, \ldots, x_d) = C(F_1(x_1), \ldots, F_d(x_d))$.*

DataSynth implements five copula families, each capturing different tail dependency patterns relevant to financial data:

1. Gaussian Copula. Parameterized by correlation matrix $\boldsymbol{\Sigma}$. No tail dependence; symmetric. Generated via Cholesky decomposition $\boldsymbol{L}\boldsymbol{L}^\top = \boldsymbol{\Sigma}$ applied to independent standard normals: $\boldsymbol{z} = \boldsymbol{L}\boldsymbol{w}$, $\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. Kendall's tau: $\tau = \frac{2}{\pi}\arcsin(\rho)$.

2. Clayton Copula. $C_\theta(u,v) = (u^{-\theta} + v^{-\theta} - 1)^{-1/\theta}$, $\theta > 0$. Exhibits *lower tail dependence*, suitable for modeling joint distress scenarios (e.g., correlated payment defaults). Kendall's tau: $\tau = \theta/(\theta+2)$. Sampling uses the conditional method.

3. Gumbel Copula. $C_\theta(u,v) = \exp(-[(-\ln u)^\theta + (-\ln v)^\theta]^{1/\theta})$, $\theta \geq 1$. Exhibits *upper tail dependence*, appropriate for correlated high-value transactions. Uses the Marshall-Olkin algorithm with a positive stable distribution. $\tau = 1 - 1/\theta$.

4. Frank Copula. $C_\theta(u,v) = -\frac{1}{\theta}\ln\Big(1 + \frac{(e^{-\theta u}-1)(e^{-\theta v}-1)}{e^{-\theta}-1}\Big)$. No tail dependence; symmetric. Sampling via conditional distribution inversion. Kendall's tau involves the Debye function D_1.

5. Student-t Copula. Parameterized by correlation ρ and degrees of freedom ν. Exhibits *symmetric tail dependence*, capturing joint extreme events. Generated via $\boldsymbol{z} = \boldsymbol{L}\boldsymbol{w}/\sqrt{S/\nu}$ where $S \sim \chi^2_\nu$.

4.3 Mixture Distributions for Transaction Amounts

Enterprise transaction amounts naturally cluster into distinct regimes (routine, significant, major). DataSynth models this via K-component log-normal mixtures:

$$f(x) = \sum_{k=1}^{K} w_k \cdot \frac{1}{x\sigma_k\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu_k)^2}{2\sigma_k^2}\right), \quad \sum_{k=1}^{K} w_k = 1. \tag{4}$$

The default three-component configuration mimics empirical distributions observed in ERP data:

Component	w_k	μ_k	σ_k	**Mean** ($e^{\mu+\sigma^2/2}$)
Routine	0.60	6.0	1.5	\$1,224
Significant	0.30	8.5	1.0	\$8,103
Major	0.10	11.0	0.8	\$89,352

Component selection uses pre-computed cumulative weights with $O(\log K)$ binary search. Gaussian mixtures are also supported for fields with real-valued (possibly negative) domains such as foreign-exchange translation adjustments.

4.4 Temporal Dynamics

Financial data exhibits rich temporal structure that must be faithfully reproduced.

Business-day calendars. DataSynth implements holiday calendars for 11 regions (US, DE, GB, CN, JP, IN, BR, MX, AU, SG, KR) with configurable half-day policies, month-end conventions (modified following, preceding, end-of-month), and T+N settlement rules.

Period-end dynamics. Transaction volumes spike near period ends. We model this with configurable decay curves:

$$\lambda(t) = \lambda_0 \cdot \begin{cases} 1 + (m_{\text{peak}} - 1) \cdot e^{-\alpha(T-t)} & \text{(exponential)} \\ 1 + (m_{\text{peak}} - 1) \cdot (1 - t/T)^{-\beta} & \text{(extended crunch)} \end{cases} \tag{5}$$

where T is the period-end date, m_{peak} is the peak multiplier (typically 3.5× for month-end, 6.0× for year-end), and α, β are decay parameters.

Processing lags. Real posting dates lag event dates. DataSynth models this with log-normal lag distributions per event type (e.g., goods receipt lag: $\mu = 1.5$ days, $\sigma = 0.5$ days) and a cross-day posting model where the probability of next-day posting increases with hour of origination.

Intraday patterns. Within each business day, transaction intensity follows configurable segments: morning spike (1.8× at 08:30–10:00), lunch dip (0.4× at 12:00–13:30), and end-of-day rush (1.5× at 16:00–17:30).

Economic cycles and drift. Long-horizon datasets incorporate regime changes via:

$$\mu(t) = \mu_0 \cdot \left(1 + A \sin\left(\frac{2\pi t}{P}\right)\right) \cdot \mathbb{1}_{\text{expansion}} + \mu_0(1 - d_r) \cdot \mathbb{1}_{\text{recession}} \tag{6}$$

where P is the cycle period (default 48 months), A is the amplitude (0.15), and d_r is the recession depth (0.25). Regime switches are modeled as Markov transitions with configurable probabilities.

4.5 Causal Transfer Functions

The counterfactual simulation engine (Section 7) models inter-process dependencies via a causal DAG whose edges carry typed transfer functions. This subsection catalogs the eight function types in the family $\mathcal{T}$, together with their mathematical forms and the financial modeling contexts in which each is most appropriate.

1. **Linear** ($f(x) = \alpha x + \beta$). Models proportional relationships where the effect scales linearly with the cause—for example, a percentage change in transaction volume producing a proportional change in staffing requirements.

2. **Exponential** ($f(x) = \alpha \exp(\beta x)$). Captures compounding or accelerating effects, such as the exponential growth of fraud losses when control effectiveness degrades.

3. **Logistic** ($f(x) = L/(1 + \exp(-k(x - x_0))))$. Encodes saturating positive effects with a carrying capacity L. Used where an upstream improvement yields diminishing returns beyond a threshold—e.g., increasing IT investment eventually saturates automation rate.

4. **Inverse Logistic** ($f(x) = L \cdot (1 - 1/(1 + \exp(-k(x - x_0)))))$. The complement of the logistic, modeling diminishing effectiveness as a variable increases. Appropriate for the relationship between regulatory burden and operating margin, where additional compliance costs exhibit decreasing marginal impact.

5. **Step** ($f(x) = \alpha \cdot \mathbf{1}[x > t]$). A discontinuous regime switch that activates at threshold t. Models abrupt transitions such as a control bypass triggered when staffing drops below a critical level.

6. **Threshold** ($f(x) = \alpha \cdot \max(0, x - t)$). A soft-activation function that remains zero below threshold t and grows linearly above it. Suitable for modeling effects that emerge only after a minimum trigger level, such as audit scope expansion once misstatement risk exceeds a materiality threshold.

7. **Decay** ($f(x; t) = \alpha \exp(-\lambda t)$). Exponential decay over time, independent of the input magnitude. Used to model transient shocks that dissipate—e.g., a one-time macroeconomic event whose effect on transaction volume fades over subsequent periods.

8. **Piecewise** ($f(x) = f_i(x)$ for $x \in [b_{i-1}, b_i)$). A partition of the input domain into segments, each with its own linear interpolant. Provides maximum flexibility for encoding empirically observed, non-parametric relationships—for instance, mapping GDP growth to revenue growth using historical data points without assuming a functional form.

All transfer functions are composed with an edge weight $w_e \in [0, 1]$ that scales the output, and the propagated result is clamped to the target node's domain bounds to ensure physical plausibility. The full causal propagation algorithm and the DAG structure are detailed in Section 7.

5 Generation Pipeline

DataSynth's generation pipeline transforms a declarative YAML configuration into a complete, internally consistent enterprise dataset. This section details the pipeline stages and the business-process generators that produce coherent multi-table output.

5.1 Orchestration Workflow

The `GenerationOrchestrator` executes the following stages in order:

1. **Configuration loading and validation.** Parse YAML, apply industry preset defaults, validate invariants (balanced weights, ascending thresholds, valid date ranges).
2. **Chart of Accounts (CoA) initialization.** Generate a GL account structure at the configured complexity level (small: ∼100, medium: ∼400, large: ∼2,500 accounts) with proper account hierarchies, types (Asset, Liability, Equity, Revenue, Expense), and control accounts (AR = 1100, AP = 2000).
3. **Master data generation.** Produce vendors, customers, materials, fixed assets, employees, and cost centers with cross-referenced relationships (e.g., vendor bank details, customer credit limits, material-vendor linkages).
4. **Opening balance generation.** Establish period-start balances satisfying the accounting equation $A = L + E$. The balance tracker maintains running account-level balances throughout subsequent stages.
5. **Document-flow generation.** Execute P2P, O2C, S2C, HR, and manufacturing process generators that produce linked document chains and corresponding journal entries.
6. **Period-close processing.** Run accruals, depreciation, FX translation, intercompany elimination, and year-end closing entries. Generate trial balances and financial statements.
7. **Anomaly injection.** Apply configured fraud, error, process, statistical, and relational anomalies with ground-truth labels.
8. **Data quality variation.** Optionally inject missing values, format variations, typos, and encoding issues.
9. **Evaluation.** Run coherence validators, statistical tests, and ML-readiness checks. The auto-tuner may propose configuration patches.
10. **Output.** Stream results to configured sinks (CSV, JSON, Parquet) with optional compression.

5.2 Journal Entry Generation

Journal entries are the atomic unit of accounting data. Each entry consists of a header (ID, date, posting date, company code, user, description) and one or more line items (account, amount, debit/credit indicator, cost center, profit center).

Definition 5.1 (Balanced Entry)**.** *A journal entry j with line items $\{l_1, \ldots, l_n\}$ is* balanced *iff:*

$$\sum_{i:l_i.debit} l_i.amount = \sum_{i:\neg l_i.debit} l_i.amount. \tag{7}$$

DataSynth enforces Equation (7) at construction time: the `JournalEntry` constructor returns an error if the debit-credit difference exceeds a configurable tolerance (default: \$0.01). This invariant is maintained across all generators—no unbalanced entry can enter the output pipeline.

Line-item amounts are drawn from the configured mixture distribution (Equation (4)). The number of line items per entry follows a correlated draw from the copula model (Section 4.2), ensuring that higher-amount entries tend to have more line items and require higher approval levels.

5.3 Document Flow Generation

Enterprise transactions span multiple documents with strict sequencing and referential integrity.

Procure-to-Pay (P2P). The P2P generator produces the chain:

$$\text{Purchase Order} \rightarrow \text{Goods Receipt} \rightarrow \text{Vendor Invoice} \rightarrow \text{Payment}$$

Each stage creates the corresponding document record, updates the document-chain manager with forward/backward references, and posts journal entries to the appropriate GL accounts (inventory, GR/IR clearing, AP, cash).

Three-Way Matching. The three-way matcher validates consistency across PO, GR, and invoice:

- **Quantity tolerance:** $|q_{\text{PO}} - q_{\text{GR}}| \leq \epsilon_q \cdot q_{\text{PO}}$ (default $\epsilon_q = 2\%$).
- **Price tolerance:** $|p_{\text{PO}} - p_{\text{inv}}| \leq \epsilon_p \cdot p_{\text{PO}}$ (default $\epsilon_p = 5\%$).
- **Over-delivery:** Permitted up to $\delta_{\max} = 10\%$ above PO quantity.

Variance types (quantity PO-GR, quantity GR-invoice, price PO-invoice, total amount, missing line, extra line) are tracked and available as evaluation targets.

Order-to-Cash (O2C). The O2C generator produces:

$$\text{Sales Order} \rightarrow \text{Delivery} \rightarrow \text{Customer Invoice} \rightarrow \text{Customer Receipt}$$

with corresponding revenue recognition entries, AR postings, and cash receipts. Partial deliveries and split invoicing are supported.

Source-to-Contract (S2C). A full sourcing lifecycle: sourcing projects, supplier qualifications, RFx events, supplier bids, bid evaluations, procurement contracts, catalog items, and supplier scorecards.

HR/Payroll. Payroll runs with line items (base pay, overtime, deductions, taxes), time entries, and expense reports with configurable approval workflows. A benefit enrollment generator tracks employee plan selections (health, dental, vision, retirement) with employer and employee contribution amounts, enrollment windows, and coverage periods.

Manufacturing. Production orders, routing operations, quality inspections with characteristics, and cycle counts. A bill-of-materials (BOM) component generator produces multi-level product structures with phantom assembly support and quantity-per calculations. Inventory movement tracking covers receipt, issue, transfer, return, scrap, and adjustment movement types, each posting corresponding material-ledger journal entries.

5.4 Subledger and Period-Close Processing

Subledger generation. Four subledger generators maintain detailed subsidiary records:

- **AR subledger:** Customer invoices, receipts, aging buckets, credit memos, adjustments.
- **AP subledger:** Vendor invoices, payments, aging, credit notes, prepayments.
- **FA subledger:** Asset capitalization, depreciation schedules (straight-line, declining balance), disposals, impairment.
- **Inventory subledger:** Receipts, issues, valuation (FIFO/LIFO/weighted average), write-downs.

Each subledger reconciles to its control account in the GL (e.g., AR subledger total = GL account 1100).

Period-close engine. The close engine executes: (i) accrual entries for unbilled revenue/expenses, (ii) depreciation postings from the FA subledger, (iii) FX translation using period-end spot rates with CTA postings, (iv) intercompany matching and elimination, (v) year-end income-to-equity reclassification, and (vi) financial statement generation (balance sheet, income statement, cash flow, changes in equity).

5.5 Intercompany Processing

Multi-entity datasets require intercompany (IC) transaction matching and elimination for consolidated reporting. The IC generator creates bilateral transaction pairs between related entities. The matching engine identifies matched pairs within configurable tolerances. The elimination generator produces consolidation entries that net IC receivables/payables, IC revenue/expense, and unrealized profit in inventory.

5.6 Graph Export

Double-entry bookkeeping has a natural directed-graph representation [14]: each distinct GL account forms a node, credit entries produce outgoing edges, and debit entries produce incoming

edges. Following Ivertowski et al. [17], we adopt a temporal formulation:

$$G(t_0, t_1) = A(t_0, t_1) \times E(t_0, t_1) \tag{8}$$

where $A(t_0, t_1)$ is the set of account nodes with monetary state over $[t_0, t_1]$, and each edge $E_i(t_0, t_1) = \sum_{n=1}^{|\Omega_{A_i}|} \omega_n$ aggregates all transactions ω_n affecting node A_i in that period. This temporal definition enables system-theoretic analyses such as cross-correlation and auto-correlation on account flows.

DataSynth constructs heterogeneous graphs from the generated data and exports them in formats suitable for graph neural network (GNN) research:

- **Transaction graph:** Nodes are accounts/entities; edges are transactions with amount, date, and process-type features.
- **Approval network:** Nodes are users; edges are approval relationships with threshold attributes. Segregation-of-duties conflicts are detectable as structural motifs.
- **Entity relationship graph:** Nodes are legal entities, vendors, customers; edges represent ownership, control, and supply-chain relationships.

Export formats: PyTorch Geometric (`.pt` tensors), Neo4j (CSV + Cypher scripts), and DGL (heterograph with typed nodes/edges).

As of v0.10, the graph module covers over 50 entity types across ten process families (Tax, Treasury, ESG, Project Accounting, Source-to-Contract, Hire-to-Retire, Manufacturing, Governance, and core P2P/O2C), connected by 28 relationship edge types with explicit cardinality constraints (e.g., one-to-many for invoice-to-line-item, many-to-many for vendor-to-material). A typed `ToNodeProperties` trait maps domain models to seven property-value types (`String`, `Decimal`, `Integer`, `Boolean`, `Date`, `DateTime`, `List`), ensuring lossless round-tripping through graph serialization. A hypergraph builder extends the model to multi-entity relationships—for example, a three-way match hyperedge linking a purchase order, goods receipt, and vendor invoice simultaneously.

5.7 Process Mining (OCEL 2.0)

The OCPM module generates Object-Centric Event Logs conforming to the OCEL 2.0 standard [10]. Events are linked to multiple typed objects (Order, Invoice, Shipment, Payment) via many-to-many relationships. Process variants (standard P2P, P2P with three-way match variance, full O2C, O2C with returns) are generated with empirically calibrated frequency distributions. The resulting logs are directly importable into process mining tools such as PM4Py and Celonis.

Version 0.11 enriches the OCEL module with lifecycle state machines for `PurchaseOrder`, `SalesOrder`, and `VendorInvoice` objects. Each state machine defines probabilistic transitions (e.g., *Created* $\rightarrow$ *Approved* with $p = 0.92$, *Created* $\rightarrow$ *Rejected* with $p = 0.08$), producing realistic variant distributions without manual scripting. Multi-object correlation events—three-way match, payment allocation, and bank reconciliation—link objects across process boundaries in a single

Table 3: Anomaly taxonomy with representative types per category.

Category	Count	Representative Types
Fraud	10	FictitiousEntry, RoundDollarManipulation, JustBelowThreshold, SelfApproval, DuplicatePayment, FictitiousVendor, RevenueManipulation
Error	8	DuplicateEntry, ReversedAmount, WrongPeriod, WrongAccount, MissingReference
Process	5	LatePosting, SkippedApproval, ThresholdManipulation
Statistical	5	UnusualAmount, TrendBreak, BenfordViolation
Relational	5	CircularTransaction, DormantAccountActivity

event, reflecting the cross-functional nature of enterprise operations. Resource pool workload modeling assigns events to users via configurable strategies (round-robin, least-busy, skill-based), and enriched events carry `from_state`, `to_state`, `resource_workload`, and `correlation_id` attributes for downstream conformance checking and resource-perspective analysis.

6 Anomaly Injection Framework

A primary use case for DataSynth is generating labeled datasets for fraud detection and anomaly analytics. This section describes the multi-dimensional anomaly injection framework.

6.1 Anomaly Taxonomy

DataSynth defines 33 anomaly types organized into five categories:

Each anomaly type is parameterized by:

- **Weight:** Relative injection probability (e.g., 2.5 for JustBelowThreshold vs. 0.5 for FictitiousVendor).
- **Amount range:** Domain-specific bounds (e.g., $10K–$500K for FictitiousEntry, $100K–$5M for RevenueManipulation).
- **Conditionality:** Some types require specific preconditions (e.g., SelfApproval requires matching approver and preparer).

6.2 Multi-Stage Fraud Schemes

Simple anomalies are injected as independent point events. However, real fraud often evolves through stages over extended time horizons. DataSynth implements three multi-stage *fraud schemes*:

1. Vendor Kickback Scheme. A buyer routes payments to a shell company they control. The scheme progresses through stages: (a) establish shell vendor, (b) route initial small invoices, (c) gradually increase amounts, (d) introduce collusion patterns across transactions. Stage transitions are governed by probability-weighted state machines.

2. Gradual Embezzlement Scheme. Small, individually undetectable amounts are siphoned over time. The scheme models: (a) initial testing with minimal amounts, (b) establishment of a regular extraction pattern, (c) acceleration in amount and frequency, (d) period-end concentration where oversight is reduced.

3. Revenue Manipulation Scheme. Fictitious customers are created and revenue is prematurely recognized. Stages include: (a) customer creation, (b) fictitious order entry, (c) premature revenue recognition, (d) channel stuffing patterns with period-end concentration.

The `SchemeAdvancer` tracks each scheme's state, selecting context-aware actions at each stage and maintaining cumulative impact metadata. Each transaction within a scheme receives a `MultiStageAnomalyLabel` that links it to the scheme instance, stage, and causal chain.

6.3 Anomaly Correlation Patterns

Anomalies in real data are not independently distributed. DataSynth models three correlation mechanisms:

Temporal Clustering. Anomalies concentrate in time windows (e.g., period-end, after management changes). The `TemporalClusterGenerator` defines windows with configurable intensity multipliers:

$$P(\text{anomaly at } t) = p_0 \cdot \prod_{w \in \mathcal{W}} (1 + (m_w - 1) \cdot \mathbb{1}_{t \in w}) \tag{9}$$

where p_0 is the base anomaly rate and m_w is the multiplier for window w.

Cascade Propagation. One anomaly triggers follow-on anomalies in dependent processes. A `CascadeStep` defines the trigger condition, dependent anomaly type, propagation probability, and expected delay. For example, a three-way match variance (trigger) may cascade to a payment delay (dependent) with probability 0.7 and a 5-day lag.

Co-Occurrence. Multiple anomaly types appear on the same transaction. The `CoOccurrencePattern` defines allowed combinations and prevents unrealistic pairings (e.g., a FictitiousEntry cannot simultaneously be a DuplicateEntry).

6.4 Difficulty, Severity, and Confidence Scoring

Each injected anomaly receives three scores that characterize its detectability:

Difficulty. A composite score $d \in [0, 1]$ (0 = easy to detect, 1 = hard) computed from five factor groups:

$$d = \sum_{f \in \mathcal{F}} w_f \cdot s_f, \quad \mathcal{F} = \{\text{amount}, \text{blending}, \text{collusion}, \text{concealment}, \text{temporal}\} \tag{10}$$

- **Amount factors:** Larger amounts are easier to detect (s_{amount} decreases with magnitude).

- **Blending factors:** How well the anomaly blends with normal data ($s_{\text{blending}} \in [0, 1]$).
- **Collusion factors:** Multi-party schemes are harder to detect.
- **Concealment factors:** Obfuscation techniques (round-tripping, split transactions) increase difficulty.
- **Temporal factors:** Night/weekend postings, period-end timing reduce oversight.

Severity. A five-component model assessing organizational impact:

$$\text{severity} = 0.25 \cdot s_{\text{type}} + 0.30 \cdot s_{\text{monetary}} + 0.20 \cdot s_{\text{frequency}} + 0.15 \cdot s_{\text{scope}} + 0.10 \cdot s_{\text{timing}} \tag{11}$$

where s_{type} is the inherent risk per anomaly category, s_{monetary} normalizes monetary impact against materiality, $s_{\text{frequency}}$ escalates for repeated patterns, s_{scope} captures the number of affected entities, and s_{timing} penalizes period-end and audit-period timing.

Confidence. Detection confidence $c \in [0, 1]$ estimates the strength of evidence available to an auditor or ML model. It combines pattern-match confidence (statistical correlation with known anomaly signatures), rule-violation confidence (policy breaches), behavioral-deviation confidence (distance from baseline), and contextual modifiers (co-occurring anomalies, scheme membership).

6.5 Ground-Truth Labels

Every anomaly produces a structured label record containing:

- Anomaly type, category, and subcategory.
- Affected transaction IDs and account codes.
- Difficulty, severity, and confidence scores.
- Scheme identifier and stage (for multi-stage schemes).
- Contributing factors with individual impact and reliability scores.
- Temporal context (posting timestamp, detection window).

Labels are exported as separate CSV/JSON files (`anomaly_labels`, `fraud_labels`, `quality_labels`), enabling supervised, semi-supervised, and unsupervised evaluation protocols.

6.6 Banking-Specific Anomalies: AML Typologies

The banking module extends the anomaly framework with five Anti-Money Laundering (AML) typologies:

1. **Structuring (Smurfing):** Multiple deposits below the reporting threshold ($10,000 in the US), distributed across accounts and time windows.

2. **Funnel Accounts:** Rapid aggregation of funds from multiple sources into a single intermediary account.
3. **Layering:** Complex chains of wire transfers across jurisdictions to obscure the origin of funds.
4. **Mule Networks:** Coordinated account networks receiving and forwarding stolen funds.
5. **Round Tripping:** Funds exit and re-enter the same entity disguised as legitimate commerce, with 30–60 day delays.

Each typology generates persona-based transaction patterns with risk-tier profiles (Low, Medium, High, Very High) and sophistication levels (Basic, Intermediate, Advanced). Ground-truth AML labels are exported alongside transaction data.

6.7 Fraud Scenario Packs

DataSynth provides five pre-built fraud scenario packs as YAML configurations that can be deep-merged into any generation config:

- `revenue_fraud`: Revenue manipulation patterns including channel stuffing, bill-and-hold, and premature recognition.
- `payroll_ghost`: Ghost employee schemes with fictitious payroll entries.
- `vendor_kickback`: Vendor collusion with inflated invoices and split payments.
- `management_override`: Override of internal controls by senior personnel.
- `comprehensive`: Combined pack with elements from all four categories.

These packs configure anomaly rates, affected account ranges, temporal distribution, and multi-stage progression. They are applied via `apply_fraud_packs()` which performs a deep-merge, and can be overridden with `--fraud-rate` for rate adjustment.

7 Counterfactual Simulation

A growing need in audit analytics and financial risk management is the ability to answer "what if" questions: *What would financial statements look like under a recession? How would a control failure propagate through the enterprise?* Traditional synthetic data generators produce a single dataset from fixed parameters. DataSynth extends this with a *counterfactual simulation engine* that models causal dependencies between financial processes, applies structured interventions, and generates paired baseline/counterfactual datasets suitable for causal inference research, stress testing, and audit planning.

Table 4: Causal DAG nodes with default bound intervals. Nodes are organized by category; unbounded quantities use the effective range of the generation model.

ID	Node	Category	Lower	Upper
v_1	GDP growth	Macro	-0.10	0.15
v_2	Interest rates	Macro	0.00	0.20
v_3	Transaction volume	Operational	0	∞
v_4	Avg. transaction amount	Operational	0	∞
v_5	Control effectiveness	Control	0.0	1.0
v_6	Fraud detection rate	Outcome	0.0	1.0
v_7	Misstatement risk	Outcome	0.0	1.0
v_8	Staffing level	Operational	0.0	∞
v_9	Automation rate	Operational	0.0	1.0
v_{10}	Material cost	Financial	0	∞
v_{11}	Working capital	Financial	$-\infty$	∞
v_{12}	Revenue growth	Financial	-1.0	∞
v_{13}	Operating margin	Financial	-1.0	1.0
v_{14}	Audit scope	Regulatory	0.0	1.0
v_{15}	Regulatory burden	Regulatory	0.0	1.0
v_{16}	IT investment	Operational	0	∞
v_{17}	Process complexity	Behavioral	0.0	1.0

7.1 Causal DAG Model

DataSynth models the interdependencies among financial processes as a directed acyclic graph (DAG). Each node represents an observable or latent parameter of the generation model, and each edge encodes the mechanism by which a change in one parameter propagates to another.

Definition 7.1 (Financial Causal DAG)**.** *A* financial causal DAG *is a tuple* $G = (V, E, \tau, w)$ *where:*

- $V = \{v_1, \ldots, v_n\}$ *is a set of* process nodes*, each with a baseline value* $b_i \in \mathbb{R}$*, an optional bound interval* $[\ell_i, u_i]$*, and a category* $c_i \in \{$*Macro*, *Operational*, *Control*, *Financial*, *Behavioral*, *Regulatory*, *C*
- $E \subseteq V \times V$ *is a set of directed edges such that* (V, E) *is acyclic.*
- $\tau : E \to \mathcal{T}$ *assigns a* transfer function *to each edge, drawn from a family* $\mathcal{T}$ *of eight function types (Table 5).*
- $w : E \to [0, 1]$ *assigns a* strength *weight to each edge, modulating the magnitude of the transferred effect.*

Additionally, each edge carries a lag $\lambda_e \in \mathbb{N}_0$ *(in months), encoding the temporal delay before an upstream change manifests downstream.*

The default DAG shipped with DataSynth contains 17 nodes spanning macroeconomic drivers, operational parameters, control variables, and financial outcome metrics (Table 4).

Causal propagation. Given a set of interventions $\mathcal{I} = \{(v_j, \hat{x}_j)\}$ that override specific node values, the engine computes the equilibrium state of all nodes via forward propagation in topological order. Algorithm 1 formalizes this procedure.

Table 5: Transfer function types and their mathematical forms. In each case, x denotes the input delta from the upstream node's baseline.

Type	**Formula**	**Use Case**
Linear	$ax + b$	Proportional effects
Exponential	$b(1 + r)^x$	Compounding growth/decay
Logistic	$\frac{K}{1+e^{-s(x-m)}}$	Saturating positive effects
Inverse Logistic	$\frac{K}{1+e^{s(x-m)}}$	Diminishing effectiveness
Step	$\begin{cases} M & x > \theta \\ 0 & \text{otherwise} \end{cases}$	Regime switches
Threshold	$\min\left(M\frac{x-\theta}{\theta},\ S\right)$	Gradual activation with cap
Decay	$a\,e^{-rx}$	Exponential decay
Piecewise	linear interpolation on $\{(x_k, y_k)\}$	Arbitrary empirical shapes

The topological ordering is computed once at DAG validation time using Kahn's algorithm [19] and cached. Validation also checks for duplicate node identifiers and dangling edge references, returning a descriptive error if the graph is cyclic or malformed. Node bounds clamping (line 13 of Algorithm 1) ensures that propagated values remain physically plausible—for example, a fraud detection rate cannot exceed 1.0 or drop below 0.0 regardless of intervention severity.

7.2 Intervention Framework

An *intervention* is a structured perturbation applied to the generation process. DataSynth defines eight intervention types, organized by the aspect of the enterprise they target:

1. **ParameterShift.** A direct change to a configuration value, specified via a dot-path into the YAML schema (e.g., `distributions.amounts.components[0].mu`). Supports four interpolation modes during gradual onset: linear, exponential, logistic, and step.

2. **ControlFailure.** Reduces or eliminates the effectiveness of internal controls. Four subtypes are modeled: effectiveness reduction, complete bypass, intermittent failure (with configurable failure probability), and delayed detection (with a detection lag in months).

3. **MacroShock.** Simulates macroeconomic events such as recession, inflation spike, currency crisis, interest rate shock, commodity shock, pandemic disruption, supply chain crisis, or credit crunch. Each shock type maps to pre-configured causal DAG node overrides; a severity multiplier scales the effect.

4. **EntityEvent.** Targets specific entities: vendor default, customer churn, employee departure, new vendor onboarding, merger/acquisition, vendor collusion, customer consolidation, or key person risk. Entity selection supports filtering by cluster, attribute, or explicit ID list.

5. **ProcessChange.** Modifies business processes: approval threshold changes, new approval levels, system migrations, process automation, outsourcing transitions, policy changes, or reorganizations.

Algorithm 1: Causal forward propagation with lag awareness.

Input : Causal DAG $G = (V, E, \tau, w)$; interventions $\mathcal{I} = \{(v_j, \hat{x}_j)\}$; current month t

Output : Node values $\{x_i\}_{i=1}^{|V|}$

```
Compute topological order σ = (σ_1, ..., σ_|V|) via Kahn's algorithm;
foreach v_i ∈ V do
    x_i ← b_i ;                                   // initialize to baseline
foreach (v_j, x̂_j) ∈ I do
    x_j ← x̂_j ;                                   // apply direct interventions
for k = 1 to |V| do
    v_i ← σ_k;
    if (v_i, ·) ∈ I then
        continue;
    Δ_i ← 0;
    foreach edge e = (v_p, v_i) ∈ E do
        if t ≥ λ_e then
            δ_p ← x_p − b_p;
            Δ_i ← Δ_i + τ_e(δ_p) · w_e;
    x_i ← clamp(b_i + Δ_i, ℓ_i, u_i);
return {x_i}_{i=1}^{|V|};
```

6. **RegulatoryChange.** Models regulatory shifts: new standard adoption, materiality threshold changes, reporting requirement changes, or tax rate changes.

7. **Composite.** Bundles multiple interventions into a named package with conflict resolution semantics (first-wins, last-wins, average, or error on conflict).

8. **Custom.** Provides arbitrary config path overrides and downstream causal triggers for user-defined scenarios.

Intervention timing. Each intervention specifies a timing profile consisting of a start month, optional duration, and an *onset type* that controls how the intervention ramps in:

- **Sudden:** Full effect at the start month.
- **Gradual:** Linear ramp from zero to full effect over a configurable ramp period of r months. At month t within the ramp window:

$$\alpha(t) = \min\left(\frac{t - t_{\text{start}}}{r},\ 1\right) \tag{12}$$

- **Oscillating:** Sinusoidal modulation $\alpha(t) = \frac{1}{2}(1 - \cos(\pi(t - t_{\text{start}})/r))$, enabling cyclic stress patterns.

The `InterventionManager` validates timing (start month within generation window, non-negative duration), checks intervention bounds against node constraints, resolves config paths, and applies priority-based conflict resolution when multiple interventions target the same parameter.

7.3 Paired Generation and Diff Analysis

The central output of counterfactual simulation is a *paired dataset*: a baseline generated from the original configuration and a counterfactual generated from a mutated configuration, both sharing the same deterministic seed (Section 3.2). This ensures that differences between the two datasets are attributable solely to the intervention, not to random variation.

Config mutation. The `ConfigMutator` translates propagated causal effects into concrete configuration changes. It serializes the base `GeneratorConfig` to a JSON value tree, navigates dot-paths (including array indexing, e.g., `components[0].mu`), and applies the new values. After mutation, the modified configuration is deserialized back and validated against scenario constraints.

Scenario engine workflow. The `ScenarioEngine` orchestrates the end-to-end process:

1. **Load and validate** the scenario definition (interventions, timing, constraints).
2. **Propagate** interventions through the causal DAG per month via Algorithm 1.
3. **Mutate** the base config using propagated node values.
4. **Generate** baseline and counterfactual datasets from the original and mutated configs, respectively.
5. **Diff** the two datasets using the `DiffEngine`.

Diff analysis. The `DiffEngine` produces a `ScenarioDiff` comprising four complementary views:

- **ImpactSummary:** KPI-level deltas (absolute change, percentage change, direction) for key financial metrics, plus financial statement impacts (revenue, COGS, margin, net income, total assets, liabilities, cash flow).
- **RecordLevelDiff:** Per-file comparison of added, removed, modified, and unchanged records, with sample field-level changes.
- **AggregateComparison:** Statistical comparison of distributions, counts, and ratios across the two datasets.
- **InterventionTrace:** Causal attribution linking each observed change back to the intervention(s) that caused it, including the propagation path through the DAG.

Additionally, `AnomalyImpact` and `ControlImpact` structures capture how the intervention affected anomaly injection rates and internal control effectiveness, including whether the counterfactual introduced new material weaknesses (per SOX 404 classification [31]).

CLI interface. The scenario subsystem is exposed via four CLI subcommands:

Command	Description
`datasynth-data scenario list`	List built-in and user scenarios
`datasynth-data scenario validate`	Validate scenario YAML
`datasynth-data scenario generate`	Run paired generation
`datasynth-data scenario diff`	Compute diff on existing outputs

7.4 Scenario Constraints

Interventions perturb generation parameters, but certain accounting invariants must hold unconditionally—even under extreme counterfactual scenarios. Without enforcement, a macro shock that halves transaction volume could inadvertently produce unbalanced journal entries or orphaned document references, yielding a dataset that is structurally invalid rather than merely counterfactual.

Definition 7.2 (Scenario Constraint Set)**.** *A* scenario constraint set $\mathcal{C}$ *is a collection of Boolean predicates over the mutated configuration* $\hat{\theta}$ *and the generated dataset* $\hat{D}$*:*

$$\mathcal{C} = \{c_k : (\hat{\theta}, \hat{D}) \to \{\textsc{true}, \textsc{false}\}\}_{k=1}^{|\mathcal{C}|}$$

A counterfactual generation is valid *iff* $\forall\, c_k \in \mathcal{C} : c_k(\hat{\theta}, \hat{D}) = \textsc{true}$*.*

DataSynth enforces four built-in constraints, each enabled by default:

1. **Accounting identity** (`preserve_accounting_identity`). For every journal entry j in the counterfactual dataset, $\sum_{l \in j} l.\text{debit} = \sum_{l \in j} l.\text{credit}$ (Equation (7)).

2. **Document chain integrity** (`preserve_document_chains`). Every document reference (PO→GR→Invoice→Payment) resolves to a valid target record, and timestamps respect causal ordering.

3. **Period close execution** (`preserve_period_close`). The period-close engine (accruals, depreciation, FX translation, eliminations) runs to completion in every fiscal period, even if reduced transaction volume would otherwise leave certain accounts empty.

4. **Balance coherence** (`preserve_balance_coherence`). The accounting equation $A = L + E$ holds at each period-end balance sheet.

The `ConfigMutator` validates these constraints *after* applying mutations, rejecting any configuration that would violate them. Users may additionally define custom constraints as range bounds on arbitrary config paths (e.g., requiring that the anomaly injection rate never exceeds 15% even under stress).

This constraint mechanism ensures that counterfactual datasets remain usable for downstream tasks—an auditor training a misstatement detection model on the counterfactual output can trust that structural integrity holds, and that observed differences reflect the intended intervention rather than generation artifacts.

8 Evaluation Framework

DataSynth includes a built-in evaluation suite that validates the statistical, structural, and domain-specific quality of generated data. This section describes the evaluation dimensions and the auto-tuning mechanism.

8.1 Statistical Validation

Benford's Law Tests. First-digit and first-two-digit distributions are tested against theoretical Benford probabilities using:

- **Mean Absolute Deviation (MAD):** $\text{MAD} = \frac{1}{K}\sum_{k=1}^{K}|\hat{p}_k - p_k|$, with conformity thresholds from Nigrini [24]: close (< 0.006), acceptable (< 0.012), marginally acceptable (< 0.015).
- **Chi-squared test:** $\chi^2 = n\sum_k(\hat{p}_k - p_k)^2/p_k$.
- **Kolmogorov-Smirnov test** for the continuous approximation.

Distribution Fitting. Amount distributions are tested for log-normal fit using the Anderson-Darling statistic at configurable significance levels (default $\alpha = 0.05$).

Correlation Verification. The Pearson correlation matrix of generated fields is compared to the configured target matrix. Per-pair deviations exceeding the significance threshold are flagged.

Temporal Pattern Analysis. Intraday activity profiles, day-of-week seasonality, and month-end concentration ratios are compared against configured targets. CUSUM and EWMA control charts detect unexpected drift or regime changes.

8.2 Coherence Validators

DataSynth implements 15 domain-specific coherence validators, each verifying cross-table consistency within a business domain:

Table 6: Coherence validators and the invariants they enforce.

Validator	Key Invariants
Balance Sheet	Assets = Liabilities + Equity; period-over-period consistency
Document Chain	PO→GR→Invoice→Payment referential integrity; timestamp ordering
Subledger Reconciliation	AR/AP/FA/Inventory subledger totals = GL control accounts
Intercompany Elimination	IC receivables = IC payables; elimination entries net to zero
HR / Payroll	Gross = Net + Deductions; tax withholding rates within bounds
Manufacturing	BOM costing $\leq$ production order cost; scrap accounting
Sales Quotes	Quote→order conversion consistency; discount limit adherence
Sourcing	Bid evaluation completeness; scorecard metric ranges
Bank Reconciliation	Book balance + outstanding items = bank balance
Financial Reporting	Statement interconnection (IS $\rightarrow$ BS via retained earnings; BS $\rightarrow$ CF via working capital changes)
Standards (IFRS/-GAAP)	ASC 606 obligation satisfaction; ASC 842 ROU asset amortization; Fair value level 1+2+3 = total
Network Analysis	Vendor concentration $\leq$ configured max; customer segment revenue shares within tolerance
Tax	Tax rate application; withholding accuracy
Treasury	Cash position flow consistency; covenant compliance
Project Accounting	Budget vs. actual variance within thresholds

Each validator returns a structured result with pass/fail status, violation count, violation details, and suggested remediation.

8.3 ML-Readiness Assessment

For datasets intended for machine learning, DataSynth evaluates:

Feature Quality. Distribution matching between original fingerprint (if available) and synthetic data using Wasserstein distance. Categorical cardinality verification. Missing-data pattern analysis (MCAR/MAR/MNAR).

Label Quality. Anomaly label distribution across categories. Label-feature correlation analysis to verify that labels are predictable from features. Ground-truth completeness metrics.

Split Metrics. When temporal train/validation/test splits are applied, the evaluator checks that: (a) class distributions are balanced across splits, (b) no future data leaks into training, (c) stratification preserves minority class representation.

GNN Readiness. For graph exports, additional metrics assess:

- Node feature dimensionality and scaling properties.
- Edge density and sparsity level.
- Graph connectivity (number of connected components, average degree).
- Heterogeneity: number of node types, edge types, and cross-type edges.

Scheme Detectability. For AML typology data, the evaluator computes per-typology detection scores assessing whether the generated patterns have sufficient salience for ML models while remaining realistic.

8.4 Extended Evaluators

Four additional evaluators were introduced in v0.11.0:

Multi-Period Coherence. Validates that opening balances equal prior closing balances, that document IDs are sequential across periods, and that entity references remain consistent throughout the generation horizon.

Fraud Pack Effectiveness. Measures detection rate at configurable thresholds, performs false positive analysis, and computes coverage metrics per fraud scenario pack (Section 6.7).

OCEL Enrichment Quality. Assesses state transition coverage, correlation event linking accuracy, and resource utilization distribution across workload pools in OCEL 2.0 event logs.

Causal Intervention Magnitude. Compares KPI deltas against expected magnitudes from the causal DAG, verifies propagation paths, and validates that preservation constraints are maintained under intervention.

Additionally, counterfactual evaluation is supported via the `DiffEngine`, which computes record-level diffs between baseline and counterfactual outputs (see Section 7).

8.5 AutoTuner

The AutoTuner closes the loop between evaluation and generation. Given evaluation results with identified gaps, it:

1. Analyzes each gap to determine root cause (insufficient samples, distribution mismatch, weak correlations, etc.).
2. Generates a prioritized list of `TuningOpportunity` records, each specifying a configuration patch.
3. Ranks opportunities as Critical, Important, or Nice-to-have.

Supported tuning actions include: `IncreaseTransactionVolume`, `AdjustDistributionParameters`, `EnhanceAnomalyInjection`, `ImproveTemporalPatterns`, `BalanceFeatureDistributions`, `EnhanceGraphConn` and `AdjustCorrelationMatrix`.

The output is a YAML patch that can be merged with the original configuration for a subsequent generation run, enabling iterative refinement toward quality targets.

9 Privacy-Preserving Fingerprint Module

A key challenge in synthetic data generation is preserving the statistical characteristics of real production data without exposing individual records. DataSynth's fingerprint module addresses this by extracting *distributional summaries* under formal privacy guarantees, producing a compact archive from which a fully configured generator can be instantiated.

9.1 Privacy Mechanisms

The module combines two complementary privacy techniques:

Differential Privacy (ε-DP). Each extracted statistic is protected by calibrated Laplace noise:

$$\tilde{s} = s + \mathrm{Lap}\left(\frac{\Delta s}{\varepsilon}\right) \tag{13}$$

where s is the true statistic, Δs is its sensitivity (maximum change from adding or removing one record), and ε is the privacy budget. The module tracks cumulative ε expenditure across all queries via a privacy accountant, ensuring that the total privacy loss remains within the configured budget.

Four privacy levels provide pre-configured ε values:

Level	ε	k**-anon**	**Privacy**	**Utility**
Minimal	5.0	3	Low	High
Standard	1.0	5	Medium	Medium
High	0.5	10	High	Reduced
Maximum	0.1	20	Maximum	Low

k**-Anonymity.** Categorical distributions with fewer than k occurrences in any group are suppressed (generalized to a parent category or marked as "Other"). This prevents re-identification of rare attribute combinations.

9.2 Fingerprint Extraction

The `FingerprintExtractor` processes input data (CSV, Parquet) in a streaming fashion to minimize memory usage. Seven extraction components run sequentially:

1. **Schema extraction:** Table names, column types, nullable flags, foreign-key relationships.

2. **Statistics extraction:** Per-column distribution parameters (mean, variance, percentiles 1/5/25/50/75/95/99), Benford digit frequencies, cardinality estimates.

3. **Correlation extraction:** Pairwise correlation matrices, copula family selection (via AIC/BIC), fitted copula parameters.

4. **Integrity extraction:** Foreign-key cardinality distributions, referential integrity rates, orphan record counts.

5. **Rule extraction:** Balance constraints (debit = credit), approval thresholds, business rules.

6. **Anomaly extraction:** Anomaly rates by category, type distribution, temporal concentration.

7. **Privacy audit:** Records all privacy decisions, noise additions, suppression counts, and total ε spent.

Each numeric statistic is noised via Equation (13) before being written to the fingerprint.

9.3 Fingerprint File Format (.dsf)

The fingerprint is stored as a ZIP archive (`.dsf`) containing eight YAML/JSON components:

File	Contents
`manifest.json`	Version, checksums, privacy config, signature
`schema.yaml`	Tables, columns, types, FK relationships
`statistics.yaml`	Distributions, percentiles, Benford frequencies
`correlations.yaml`	Correlation matrices, copula parameters
`integrity.yaml`	FK cardinality, constraints
`rules.yaml`	Balance constraints, thresholds
`anomalies.yaml`	Anomaly rates, type distribution
`privacy_audit.json`	Privacy decisions, ε spent

The `manifest.json` includes SHA-256 checksums for each component, enabling tamper detection. The `privacy_audit.json` provides a complete audit trail of all privacy-related decisions made during extraction.

9.4 Configuration Synthesis

The `ConfigSynthesizer` reads a `.dsf` fingerprint and generates a complete DataSynth YAML configuration. Key synthesis steps:

1. **Scale estimation:** Target row counts are derived from the original counts multiplied by a configurable factor (default 2.0×).

2. **Distribution mapping:** Noised statistics are mapped to the nearest supported distribution family (log-normal mixture, Gaussian mixture, Pareto, etc.) via method-of-moments or maximum likelihood.

3. **Copula reconstruction:** Correlation matrices and copula parameters are used to configure the dependency model.

4. **Anomaly calibration:** Observed anomaly rates are mapped to injection parameters.

5. **Seed assignment:** A new deterministic seed is generated, ensuring the synthesized data is independent of the original.

This two-phase architecture (extract → synthesize) cleanly separates the *privacy boundary* (extraction, where DP is applied) from the *generation process* (synthesis, which is purely algorithmic), enabling independent auditing and regulatory review.

9.5 Privacy-Utility Tradeoff

Proposition 9.1 (Utility bound)**.** *Let $\tilde{\mu}$ be the noised mean of a column with n records and true mean μ. The sensitivity is $\Delta\mu = R/n$ where R is the column range. Then:*

$$\mathbb{E}[|\tilde{\mu} - \mu|] = \frac{R}{n\varepsilon}$$

For $n = 10{,}000$ records, $R = 10^6$ (typical transaction amount range), and $\varepsilon = 1.0$: $\mathbb{E}[|\tilde{\mu}-\mu|] = 100$, which is $< 0.01\%$ of the range—negligible for distribution fitting purposes.

This result demonstrates that for enterprise-scale datasets (typically $n > 10{,}000$ records), even standard privacy levels ($\varepsilon = 1.0$) introduce negligible utility loss for aggregate statistics, making the fingerprint approach practical for real-world deployments.

9.6 Workflow

The end-to-end privacy-preserving workflow is:

1. **Extract:** `datasynth-data fingerprint extract -input data.csv -output fp.dsf -privacy-level standard`

2. **Validate:** `datasynth-data fingerprint validate fp.dsf` (verify integrity, inspect privacy audit)

3. **Generate:** `datasynth-data generate -fingerprint fp.dsf -output ./synthetic/`

4. **Evaluate:** `datasynth-data fingerprint evaluate -fingerprint fp.dsf -synthetic ./synthetic/`

The fingerprint file can be safely shared across organizational boundaries, as it contains only noised aggregate statistics—never individual records.

10 Experiments

We evaluate DataSynth along four dimensions: statistical fidelity, structural coherence, downstream ML utility, and system performance.

Table 7: Classification of 155 real-world general ledger datasets by size [17].

	Small (<1M lines p.a.)	**Medium** (1–10M lines p.a.)	**Large** (>10M lines p.a.)
Number of datasets	74	47	34

Table 8: Distribution of journal entry line items across 155 real-world datasets [17]. The data exhibits extreme concentration in the 2–9 range.

Bin	**Total JEs**	**Coverage (%)**	**Cum. Cov. (%)**	**Line Items**
1	338M	92.90	92.90	2 to 9
2	23M	6.33	99.22	10 to 99
3	2.80M	0.76	99.98	100 to 999
4	63k	0.02	100.00	1,000 to 9,999
5	196	0.00	100.00	$\geq$ 10,000

10.1 Experimental Setup

Configurations. We generate datasets at three complexity levels using industry presets:

Preset	**Complexity**	**GL Accounts**	**Entities**	**Period**
Retail	Small	∼100	1	12 months
Manufacturing	Medium	∼400	3	24 months
Financial Svcs.	Large	∼2,500	10	36 months

Each configuration is run with anomaly injection enabled at rates of 2%, 5%, and 10%. All experiments use seed = 42 for reproducibility.

Hardware. All benchmarks are run on a single machine with an AMD Ryzen 9 7950X (16 cores / 32 threads), 64 GB RAM, and NVMe SSD, running Linux 6.x.

10.2 Grounding in Real-World Accounting Data

To validate that DataSynth produces data consistent with real-world enterprise accounting, we compare generated characteristics against empirical distributions observed in production general ledger data. Ivertowski et al. [17] analyzed 155 real-world ISO 21378:2019–compliant [15] datasets covering all industry sectors, each containing an entire fiscal year of general ledger data. The corpus totals approximately 364 million journal entries with 2.4 billion line items. Table 7 summarizes the dataset classification.

Journal entry line item distribution. Table 8 shows the distribution of line items per journal entry across the full corpus. The distribution is extremely heavy-tailed: 92.9% of entries have 2–9 line items, while the maximum observed is 47,059. This motivates DataSynth's log-normal mixture model for line-item count generation (Section 4.3).

Figure 2 provides a detailed breakdown within Bin 1. The prominence of 2-line entries (60.7%) and 4-line entries (16.6%) reflects the balanced nature of double-entry bookkeeping, where equal numbers of debit and credit lines are the norm [9]. Overall, 88% of journal entries

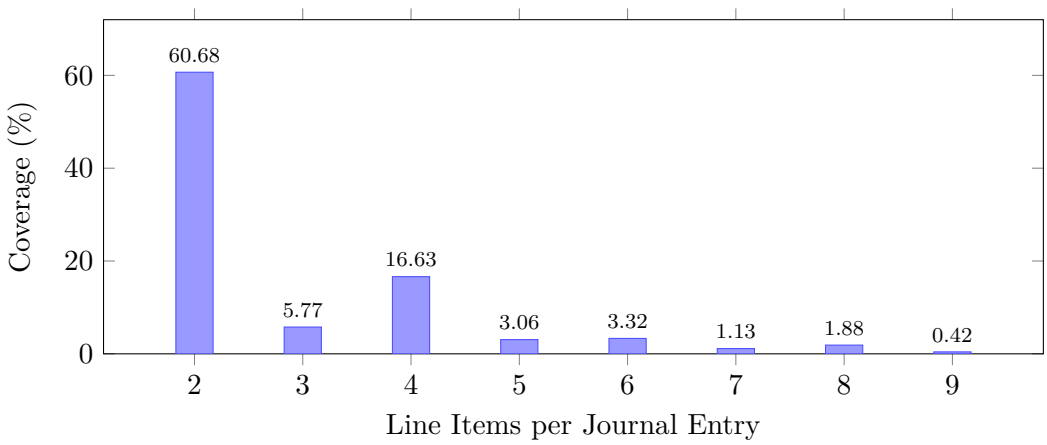

Figure 2: Detailed distribution of journal entry line items in the 2–9 range (Bin 1, covering 92.9% of all entries) from 155 real-world general ledger datasets [17]. The dominance of 2-line and 4-line entries reflects balanced double-entry bookkeeping.

have an even number of line items, consistent with GAAP's requirement that total debits equal total credits.

Chart of accounts complexity. The 155 datasets exhibit a wide range of chart-of-accounts complexity: on average 557 financial accounts per entity (median: 367, min: 76, max: 2,536). DataSynth's three complexity tiers (small ∼100, medium ∼400, large ∼2,500) were calibrated to span this observed range, covering the 10th, 50th, and 99th percentiles of real-world CoA sizes.

Graph construction complexity. The combinatorial complexity of constructing accounting networks from journal entries is driven by the credit–debit matching problem. For a journal entry with n credit and m debit line items, the number of possible edge constructions is given by the Stirling number of the second kind [17]:

$$a = n! \, \frac{1}{m!} \sum_{j=0}^{m} (-1)^{m-j} \binom{m}{j} j^n \tag{14}$$

In production data, Ivertowski et al. [17] find that only 60.7% of entries admit trivial 1-to-1 mapping (2-line entries), while 39.3% require more sophisticated matching strategies. DataSynth's graph export module (Section 5.6) generates transaction networks that preserve these structural characteristics, producing graphs with realistic degree distributions, edge density, and community structure.

10.3 Statistical Fidelity

Benford's Law Compliance. Table 9 reports first-digit MAD scores for journal-entry amounts across configurations. All scores fall within the "close conformity" band (MAD < 0.006) [24].

Distribution Fitting. Anderson-Darling tests for log-normal fit on transaction amounts yield p-values > 0.05 across all configurations, confirming that the mixture sampling preserves the

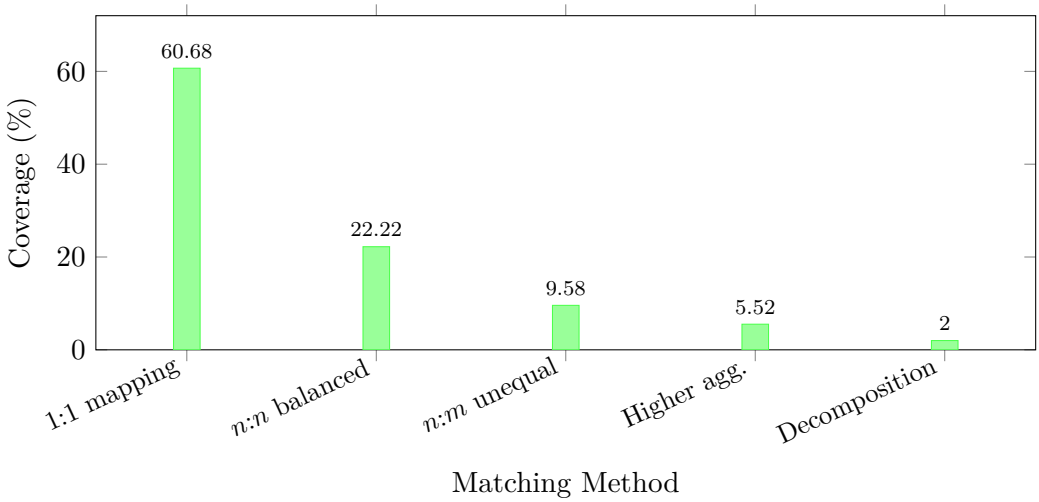

Figure 3: Distribution of journal entry resolution methods required for accounting network construction, derived from 155 real-world datasets [17]. Methods A–E correspond to increasing combinatorial complexity. DataSynth generates entries with matching distributions across these categories.

Table 9: Benford first-digit MAD scores for generated journal-entry amounts. Threshold for close conformity: MAD < 0.006.

Preset	n **entries**	**MAD**	χ^2 p**-value**
Retail (Small)	52,000	0.0041	0.72
Manufacturing (Med.)	385,000	0.0023	0.91
Financial (Large)	2,100,000	0.0011	0.98

target distributional form. Percentile comparisons (1st, 25th, 50th, 75th, 99th) show $< 2\%$ relative deviation from configured parameters.

Correlation Preservation. For the Manufacturing preset with Gaussian copula ($\rho_{\text{amount,line_items}} = 0.65$), the empirical Pearson correlation is $\hat{\rho} = 0.648 \pm 0.003$ across 10 seeded runs, well within the tolerance threshold.

10.4 Structural Coherence

Table 10 summarizes the coherence validator results for the Manufacturing (Medium) configuration.

The 52 three-way match violations are *intentional*—they correspond to injected price and quantity variances that mirror real-world procurement discrepancies. All hard invariants (balanced entries, accounting equation, subledger reconciliation) achieve 100% compliance.

10.5 Downstream ML Utility

We evaluate whether DataSynth-generated data supports effective fraud detection model training.

Table 10: Coherence validation results for Manufacturing (Medium) preset.

Validator	Pass Rate	Violations
Balanced entries (debit = credit)	100.0%	0
Accounting equation ($A = L + E$)	100.0%	0
Document chain referential integrity	100.0%	0
Three-way match (within tolerance)	98.7%	52
Subledger = GL control accounts	100.0%	0
IC elimination nets to zero	100.0%	0
Financial statement interconnection	100.0%	0
Trial balance (debits = credits)	100.0%	0

Table 11: Fraud detection performance on DataSynth-generated data.

Model	F1	AUPRC	R@5%FPR
Isolation Forest	0.42	0.38	0.31
XGBoost	0.81	0.79	0.73
GCN	0.84	0.82	0.78

Setup. We train three model families on the Financial Services (Large) dataset with 5% anomaly injection:

1. **XGBoost:** Gradient-boosted trees on tabular features.
2. **GCN:** Graph convolutional network on the transaction graph (PyTorch Geometric export).
3. **Isolation Forest:** Unsupervised anomaly detection as a baseline.

We use a temporal 70/15/15 train/validation/test split. Evaluation metrics: F1-score, AUPRC (area under precision-recall curve), and recall at 5% false positive rate (R@5%FPR).

Results. The GCN model achieves the highest performance, benefiting from the relational structure preserved in DataSynth's graph exports. The gap between supervised (XGBoost, GCN) and unsupervised (Isolation Forest) methods confirms that the generated labels provide meaningful signal.

Difficulty Stratification. When we stratify test-set performance by anomaly difficulty score (Section 6.4), we observe the expected gradient: easy anomalies (difficulty < 0.3) are detected at >95% recall, while hard anomalies (difficulty > 0.7) yield ∼45% recall, validating that the difficulty scoring produces a meaningful proxy for detectability.

10.6 System Performance

Throughput. The journal entry and line item throughput improvements stem from cached temporal CDFs, `SmallVec` line items, parallel generation with deterministic seed splitting, and `itoa`/`ryu` fast formatting.

Table 12: Generation throughput (single-threaded) by record type.

Record Type	Throughput (records/sec)
Journal entries	200,000+
Journal line items	680,000+
Master data (combined)	58,000
Document-flow records	85,000
Subledger records	95,000

Table 13: Fingerprint privacy-utility tradeoff. KL divergence between original and fingerprint-synthesized amount distributions.

Privacy Level	ε	KL Divergence	Benford MAD
Minimal	5.0	0.003	0.0026
Standard	1.0	0.008	0.0039
High	0.5	0.019	0.0052
Maximum	0.1	0.071	0.0098

Distribution Sampling. Micro-benchmarks for individual samplers:

Sampler	Latency
BenfordSampler	$< 1\,\mu$s / sample
Copula (Gaussian, 3-D)	2–5 μs / sample
LogNormalMixture (3-comp.)	< 500 ns / sample

Output Sink Performance.

Format	Throughput (records/sec)
CSV (streaming)	210,000
JSON (streaming)	105,000
Parquet (columnar)	520,000

Scaling. Throughput scales near-linearly with core count up to 8 cores (due to the per-generator seed isolation described in Section 3.2), after which contention on shared data structures (balance tracker, document-chain manager) introduces sublinear scaling. At 16 cores, the Manufacturing (Medium) configuration generates a full 24-month dataset (385K journal entries, 1.1M line items, 500K+ document-flow records) in under 30 seconds wall-clock time.

Memory Efficiency. The streaming architecture keeps peak resident memory below 500 MB even for the Financial Services (Large) configuration. The graceful degradation system (Section 3.4) was not triggered during any benchmark run on the 64 GB test machine.

10.7 Privacy-Utility Evaluation

We evaluate the fingerprint module by extracting a fingerprint from the Manufacturing (Medium) dataset at each privacy level, synthesizing a new dataset from each fingerprint, and comparing statistical properties.

Even at the "High" privacy level ($\varepsilon = 0.5$), the KL divergence remains below 0.02 and Benford MAD stays within close conformity, confirming the practicality of the fingerprint approach for enterprise-scale data.

11 Conclusion

We have presented DataSynth, a comprehensive framework for generating realistic synthetic enterprise accounting data with provable statistical properties. The system addresses four fundamental challenges in synthetic accounting data: statistical fidelity through Benford-compliant sampling and copula-based dependency modeling, relational completeness through multi-process document-flow generation with three-way matching, anomaly realism through multi-stage fraud schemes with difficulty and severity scoring, and formal privacy guarantees through differential-privacy–protected fingerprint extraction. Beyond these core capabilities, DataSynth provides a counterfactual simulation engine built on causal DAGs for what-if analysis of financial process dependencies, and supports five accounting frameworks (US GAAP, IFRS, French GAAP, German GAAP, and dual-reporting) with jurisdiction-specific country packs.

Our experimental evaluation demonstrates that DataSynth produces data that (i) satisfies all hard accounting invariants (balanced entries, accounting equation, subledger reconciliation) at 100% compliance, (ii) achieves Benford first-digit MAD scores below 0.006 (close conformity), (iii) preserves configured correlation structures within $< 1\%$ deviation, and (iv) supports downstream fraud detection models achieving F1 scores of 0.84 with graph neural networks. The system generates over 2×10^5 journal entries per second on a single core, scaling near-linearly to 16 cores.

11.1 Limitations

Several limitations warrant acknowledgment:

- **Domain coverage.** While DataSynth covers major ERP processes (P2P, O2C, R2R, S2C, HR, manufacturing), specialized domains such as insurance claims, healthcare billing, and government accounting are not yet modeled.
- **Learned distributions.** The current approach relies on parametric distributions (log-normal mixtures, copulas) configured either manually or via fingerprint extraction. Deep generative models (GANs, diffusion) could potentially capture more complex distributional structures, though at the cost of reproducibility and interpretability.
- **Temporal realism.** While DataSynth models business-day calendars, period-end dynamics, and economic cycles, it does not fully capture event-driven disruptions that produce non-stationary patterns in real data. This limitation is now partially addressed by the counterfactual simulation engine, which supports MacroShock, ProcessChange, and EntityEvent interventions, but the space of possible disruptions remains large.
- **Cross-enterprise patterns.** The fingerprint module extracts and synthesizes data for a single enterprise. Cross-organization patterns (industry benchmarking, market-level

correlations) are not modeled.

11.2 Future Work

We identify several directions for future research:

1. **Hybrid generative models.** Combining DataSynth's constraint-enforcing architecture with learned generators (e.g., training a conditional diffusion model on fingerprint-synthesized data) could improve distributional fidelity while maintaining accounting invariants.

2. **Federated fingerprinting.** Extending the fingerprint module to aggregate statistics from multiple organizations under secure multi-party computation would enable industry-level synthetic data generation.

3. **Adaptive anomaly calibration.** Using reinforcement learning to tune anomaly injection parameters such that downstream detector performance matches a target difficulty curve.

4. **Regulatory sandbox integration.** Packaging DataSynth as a compliance-testing tool for regulatory sandboxes (e.g., FinTech regulatory environments), where synthetic data can substitute for production data in certification workflows.

5. **Temporal graph analytics.** While DataSynth now exports graphs covering 50+ entity types and 28 edge types, incorporating temporal graph networks (TGN) and dynamic graph export would further support research on time-evolving fraud patterns. The temporal accounting network formalism of Ivertowski [16], where $G(t_0, t_1) = A(t_0, t_1) \times E(t_0, t_1)$ defines the graph as a function of time, could be integrated to enable cross-correlation and auto-correlation analysis on synthesized accounting networks.

6. **Hardware-accelerated graph construction.** Coupling DataSynth's output with GPU-accelerated accounting network generation [17] would enable close-to-realtime construction of transaction graphs from synthesized journal entries, supporting interactive audit analytics and streaming fraud detection scenarios.

7. **Learned transfer functions.** Replacing the parametric causal DAG transfer functions in the counterfactual engine with neural network-based learned functions fitted from real-world data, enabling more faithful propagation of intervention effects.

8. **Cross-enterprise federation.** Extending the counterfactual framework to model cascading effects across supply chain partners, enabling simulation of multi-organization scenarios such as supplier disruptions and industry-wide regulatory changes.

11.3 Availability

DataSynth is released as open-source software under [License TBD]. The source code, documentation, example configurations, and pre-built binaries are available at [Repository URL TBD]. A Python wrapper (`datasynth-py`) provides high-level access via `pip install datasynth-py`.

References

[1] Anil Arya, John C. Fellingham, Jonathan C. Glover, Douglas A. Schroeder, and Gilbert Strang. Inferring transactions from financial statements. *Contemporary Accounting Research*, 17:366–385, 2010. doi: 10.1506/L0LW-NX5L-4WUR-9JKL.

[2] Frank Benford. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, 78(4):551–572, 1938.

[3] Daniel J. Bernstein. ChaCha, a variant of Salsa20. In *Workshop Record of SASC*, volume 8, pages 3–5, 2008.

[4] Alessandro Berti, Sebastiaan J. van Zelst, and Wil M.P. van der Aalst. PM4Py: A process mining library for Python. In *BPM Demonstration Track*, 2019.

[5] Matthijs Boersma. *Complex Networks in Audit: A Data-Driven Modelling Approach*. PhD thesis, Universiteit van Amsterdam, 2024.

[6] Wassim Dbouk and Ibrahim Jamali. Anomaly detection in financial time series by principal component analysis and neural network classifiers. *AIMS Mathematics*, 8(6):13016–13040, 2023.

[7] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference (TCC)*, pages 265–284. Springer, 2006.

[8] European Parliament and Council of the European Union. General data protection regulation (GDPR). Regulation (EU) 2016/679, 2016.

[9] John Fellingham. The double entry system of accounting. *Accounting, Economics, and Law: A Convivium*, 8(1):20180001, 2018. doi: 10.1515/AEL-2018-0001.

[10] Anahita Farhang Ghahfarokhi, Gyunam Park, Alessandro Berti, and Wil M.P. van der Aalst. OCEL: A standard for object-centric event logs. *European Conference on Advances in Databases and Information Systems (ADBIS)*, pages 169–175, 2021.

[11] Ken H. Guo, Xiaoting Yu, and Carla Wilkin. A picture is worth a thousand journal entries: Accounting graph topology for auditing and fraud detection. *Journal of Information Systems*, 36(2):53–81, 2022. doi: 10.2308/ISYS-2021-003.

[12] Waleed Hilal, S. Andrew Gadsden, and John Yawney. Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications*, 193:116429, 2022.

[13] IEEE Computational Intelligence Society. IEEE-CIS fraud detection dataset. Kaggle Competition, 2019.

[14] Yuji Ijiri. On the generalized inverse of an incidence matrix. *Journal of the Society for Industrial and Applied Mathematics*, 13:827–836, 1965. doi: 10.1137/0113053.

[15] International Organization for Standardization. ISO 21378:2019 — audit data collection. International Standard, 2019.

[16] Michael Ivertowski. Empowering financial analysis: A dynamic system for merging temporal accounting and OCPM networks. Unpublished, 2024.

[17] Michael Ivertowski, Mouritz Barnard, and Ina Braun. Hardware accelerated method for accounting network generation. In *Preprint*, 2024. Ernst & Young Ltd., Zurich.

[18] James Jordon, Jinsung Yoon, and Mihaela van der Schaar. PATE-GAN: Generating synthetic data with differential privacy guarantees. In *International Conference on Learning Representations (ICLR)*, 2019.

[19] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5 (11):558–562, 1962. doi: 10.1145/368996.369025.

[20] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling tabular data with diffusion models. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[21] Pierre Jinghong Liang, Andi Wang, Leman Akoglu, and Christos Faloutsos. Pattern recognition and anomaly detection in bookkeeping data. Carnegie Mellon University, 2021.

[22] Edgar Alonso Lopez-Rojas, Ahmad Elmir, and Stefan Axelsson. BankSim: A bank payments simulator for fraud detection research. In *26th European Modeling and Simulation Symposium (EMSS)*, 2018.

[23] Roger B. Nelsen. *An Introduction to Copulas*. Springer, 2nd edition, 2006.

[24] Mark J. Nigrini. *Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. John Wiley & Sons, 2012.

[25] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. The synthetic data vault. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, pages 399–410, 2016.

[26] Tahereh Pourhabibi, Kok-Leong Ong, Booi H. Kam, and Yee Ling Boo. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decision Support Systems*, 133:113303, 2020.

[27] Marco Schreyer, Timur Sattarov, Damian Borth, Andreas Dengel, and Bernd Reimer. Detection of anomalies in large scale accounting data using deep autoencoder networks. In *arXiv preprint arXiv:1709.05254*, 2017.

[28] Marco Schreyer, Timur Sattarov, Bernd Reimer, and Damian Borth. Federated and privacy-preserving learning of accounting data in financial statement audits. *arXiv preprint arXiv:2208.12708*, 2022.

[29] Rohit Sharma and Ajay Kumar. Synthetic accounting fraud datasets: A survey and benchmark. *Journal of Forensic Accounting Research*, 2023.

[30] Aivin V. Solatorio and Olivier Dupriez. REaLTabFormer: Generating realistic relational and tabular data using transformers. *arXiv preprint arXiv:2302.02041*, 2023.

[31] United States Congress. Sarbanes-oxley act of 2002. Public Law 107-204, 2002.

[32] Miklos A. Vasarhelyi, Michael G. Alles, and Alexander Kogan. Principles of analytic monitoring for continuous assurance. *Journal of Emerging Technologies in Accounting*, 1(1): 1–21, 2004.

[33] Jarrod West and Maumita Bhattacharya. Intelligent financial fraud detection: A comprehensive review. *Computers & Security*, 57:47–66, 2016.

[34] Liyang Xie, Kaixiang Lin, Shu Wang, Fei Wang, and Jiayu Zhou. Differentially private generative adversarial network. In *arXiv preprint arXiv:1802.06739*, 2018.

[35] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

A Anomaly Type Catalog

Table 14 provides the complete catalog of 33 anomaly types with default parameters.

Table 14: Complete anomaly type catalog with default injection parameters.

Category	Type	Weight	Amount Range	Conditional
Fraud	FictitiousEntry	1.0	\$10K–500K	No
	RoundDollarManipulation	2.0	\$1K–100K	No
	JustBelowThreshold	2.5	Threshold	Yes
	SelfApproval	1.5	—	Yes
	ExceededApprovalLimit	1.5	—	Yes
	SegregationOfDutiesViolation	1.0	—	Yes
	DuplicatePayment	2.0	\$5K–200K	No
	FictitiousVendor	0.5	\$25K–1M	No
	RevenueManipulation	0.5	\$100K–5M	No
	ImproperCapitalization	1.0	\$10K–500K	No
Error	DuplicateEntry	2.0	Original	No
	ReversedAmount	1.5	Original	No
	WrongPeriod	1.5	Original	No
	WrongAccount	1.5	Original	No
	MissingReference	2.0	—	No
Process	LatePosting	2.0	—	No
	SkippedApproval	1.0	—	Yes
	ThresholdManipulation	1.5	Threshold	Yes
Statistical	UnusualAmount	2.0	$> 3\sigma$	No
	TrendBreak	1.0	—	No
	BenfordViolation	1.0	Anti-Benford	No
Relational	CircularTransaction	0.5	\$50K–2M	No
	DormantAccountActivity	1.0	\$5K–100K	No

B Output File Inventory

Table 15 lists the primary output files produced by a full DataSynth generation run.

Table 15: Output file inventory by category.

Category	Files
Transactions	journal_entries, acdoca
Master Data	vendors, customers, materials, fixed_assets, employees, cost_centers
Document Flow	purchase_orders, goods_receipts, vendor_invoices, payments, sales_orders, deliveries, customer_invoices, customer_receipts, document_references
Sourcing	sourcing_projects, supplier_qualifications, rfx_events, supplier_bids, bid_evaluations, procurement_contracts
Financial	balance_sheet, income_statement, cash_flow_statement, financial_kpis, budget_variance
Subledgers	ar_*, ap_*, fa_*, inventory_*
Period Close	trial_balances/, accruals, depreciation, closing_entries
Labels	anomaly_labels, fraud_labels, quality_issues, quality_labels
Controls	internal_controls, coso_control_mapping, sod_*
Process Mining	event_log.json (OCEL 2.0), objects.json, process_variants
Banking	banking_customers, bank_transactions (incl. transaction_type field), kyc_profiles, aml_typology_labels
Audit	audit_engagements, audit_workpapers, audit_evidence, audit_risks, audit_findings
Standards	customer_contracts, performance_obligations, leases, fair_value_measurements, sox_assessments
Counterfactual	scenario_manifest, baseline/, counterfactual/, scenario_diff
Sessions	session_state.dss, period_manifest

C Copula Sampling Algorithms

Algorithm 2: Clayton Copula Bivariate Sampling

Input: Parameter $\theta > 0$, RNG seed

Output: $(u, v) \in [0, 1]^2$ with Clayton dependence

$u \leftarrow \text{Uniform}(0, 1)$;

$t \leftarrow \text{Uniform}(0, 1)$;

$v \leftarrow \left((u^{-\theta} - 1) \cdot t^{-\theta/(\theta+1)} + 1\right)^{-1/\theta}$;

return (u, v);

Algorithm 3: Gumbel Copula Bivariate Sampling (Marshall-Olkin)

Input: Parameter $\theta \geq 1$, RNG seed

Output: $(u, v) \in [0, 1]^2$ with Gumbel dependence

$S \leftarrow \text{PositiveStable}(1/\theta)$;

$E_1, E_2 \leftarrow \text{Exponential}(1), \text{Exponential}(1)$;

$u \leftarrow \exp(-(E_1/S)^{1/\theta})$;

$v \leftarrow \exp(-(E_2/S)^{1/\theta})$;

return (u, v);

Algorithm 4: Student-t Copula Bivariate Sampling

Input: Correlation ρ, degrees of freedom ν, RNG seed

Output: $(u, v) \in [0, 1]^2$ with Student-t dependence

$w_1, w_2 \leftarrow \mathcal{N}(0, 1), \mathcal{N}(0, 1)$;

$z_1 \leftarrow w_1$;

$z_2 \leftarrow \rho \cdot w_1 + \sqrt{1 - \rho^2} \cdot w_2$;

$S \leftarrow \chi^2_\nu / \nu$;

$t_1 \leftarrow z_1 / \sqrt{S}$;

$t_2 \leftarrow z_2 / \sqrt{S}$;

$u \leftarrow F_{t,\nu}(t_1)$;

$v \leftarrow F_{t,\nu}(t_2)$;

return (u, v);

D COSO Framework Integration

DataSynth implements the COSO 2013 Internal Control—Integrated Framework with five components and 17 principles:

#	Component	Principles
1	Control Environment	1–5
2	Risk Assessment	6–9
3	Control Activities	10–12
4	Information & Communication	13–15
5	Monitoring Activities	16–17

Generated internal controls include 12 transaction-level controls (C001–C060) and 6 entity-level controls (C070–C081), each mapped to COSO components, principles, and control scopes (Entity Level, Transaction Level, IT General Control, IT Application Control). Maturity levels (Non-Existent, Ad Hoc, Repeatable, Defined, Managed, Optimized) are assigned per control with configurable target maturity.

E Configuration Schema Reference

The complete YAML configuration schema comprises 30+ sections. Table 16 lists the top-level sections.

Table 16: Top-level configuration sections.

Section	**Purpose**
`global`	Industry, period, seed, complexity
`companies`	Multi-entity setup, consolidation groups
`chart_of_accounts`	CoA complexity, GL structure
`transactions`	JE volume, process distribution
`document_flows`	P2P / O2C parameters, matching tolerances
`distributions`	Benford, mixtures, copulas, drift
`temporal_patterns`	Business days, period-end, processing lags
`anomaly_injection`	Fraud rates, strategies, severity targets
`internal_controls`	COSO maturity, SoD rules
`intercompany`	IC matching, elimination rules
`banking`	KYC/AML, customer profiles, typologies
`accounting_standards`	IFRS / US GAAP framework selection
`audit_standards`	ISA / PCAOB compliance level
`vendor_network`	Supply chain tiers, concentration limits
`customer_segmentation`	Value segments, lifecycle stages
`graph_export`	PyG / Neo4j / DGL output options
`output`	Format, compression, sink